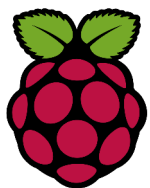


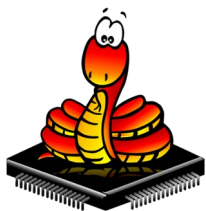


Atelier Raspi

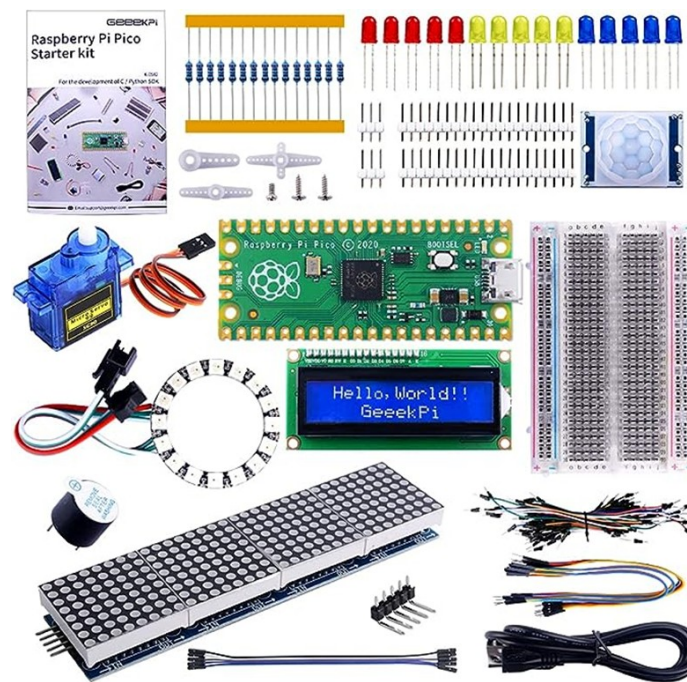
Atelier N°19 Le Jeu de Simon



Logo du Raspberry Pico



Logo du MicroPython



L'atelier a pour valeurs, le partage, l'aide, la formation, le faire et construire ensemble à partir de l'expérience des participants

Atelier N°19 Le Jeu de Simon

- Etape 1 Le jeu classique
- Etape 2 Rien ne se jette....
- Etape 3 Rappel du câblage
- Etape 4 Un programme évolutif !
- Etape 5 Version simplifiée (1/2)
- Etape 6 Version simplifiée (2/2)
- Etape 7 Code LCD à réutiliser
- Etape 8 Code Neopixel à réutiliser (1/2)
- Etape 9 Code Neopixel à réutiliser (2/2)



A19E1 Le jeu classique

JEU SIMON (Classique)

LE JEU ÉLECTRONIQUE DE MÉMOIRE LUMINEUSE ET SONORE



VUE DE DESSUS (FACE AVANT)

FONCTIONNEMENT

1. Le jeu génère une séquence de couleurs et de sons.
2. Le joueur doit répéter la séquence en appuyant sur les boutons dans le bon ordre.
3. À chaque niveau réussi, la séquence s'allonge.
4. Le jeu s'arrête en cas d'erreur (Game Over).
5. Le score (niveau atteint) est affiché.

CARACTÉRISTIQUES

- 4 BOUTONS LUMINEUX (LED)
 - – ROUGE
 - – VERTE
 - – JAUNE
 - – BLEUE
- MODES DE JEU : 1, 2, 3
- AFFICHAGE SCORE (2 CHIFFRES)
- SONORISATION (BIPS)
- FONCTION "LONGEST" (MEILLEUR SCORE)
- ALIMENTATION : 3 PILES LR14 (C)
- CONSTRUCTION ROBUSTE EN PLASTIQUE ABS

A19E2 Rien ne se jette, rien ne s'achète, tout se réutilise !

JEU SIMON (Classique)

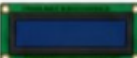
LE JEU ÉLECTRONIQUE DE MÉMOIRE LUMINEUSE ET SONORE

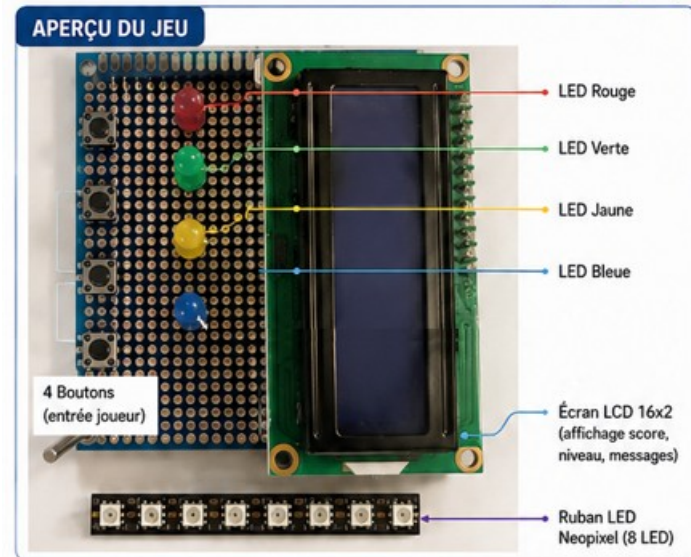


VUE DE DESSUS (FACE AVANT)



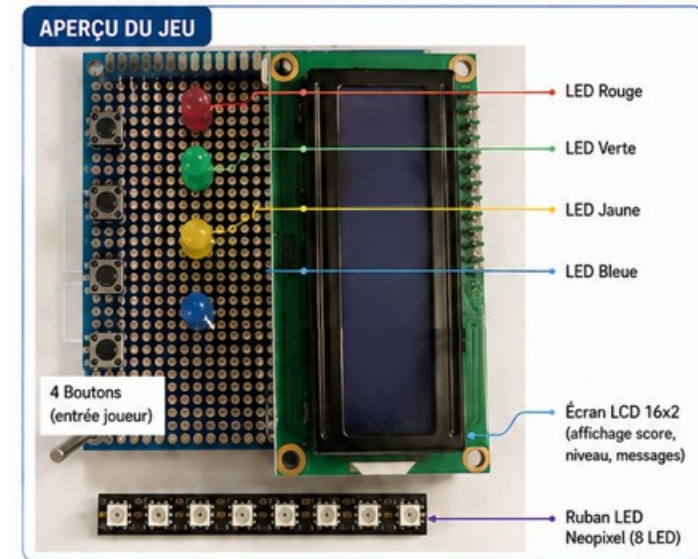
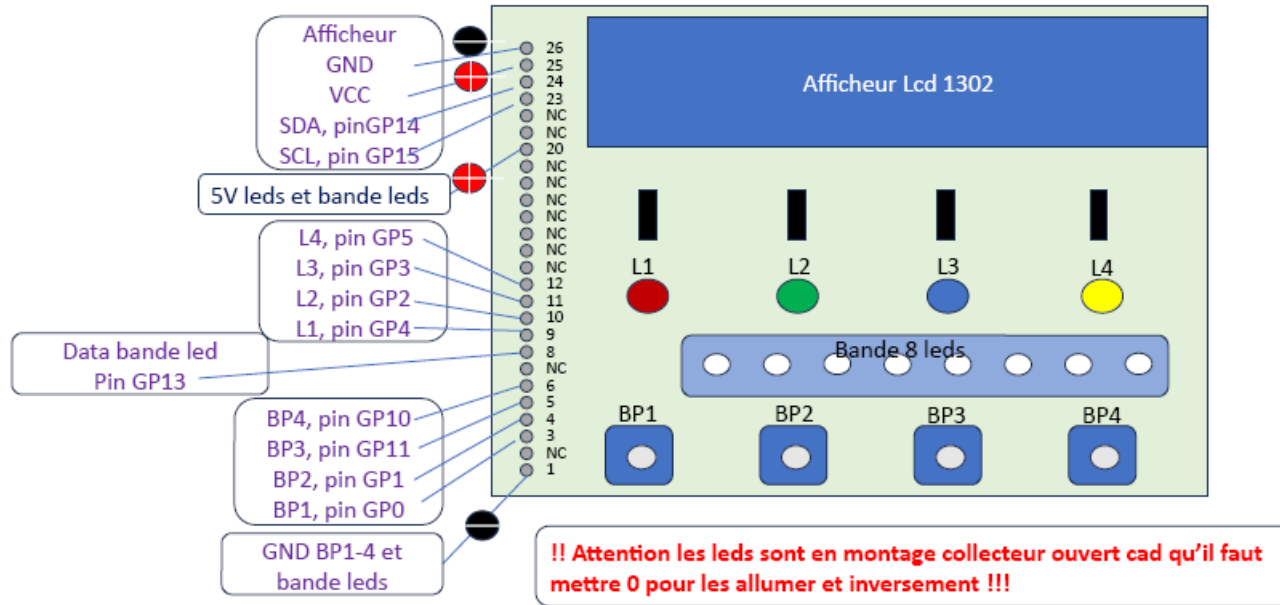
COMPOSANTS UTILISÉS

-  - 1 × Raspberry Pi Pico
-  4 × LED 5mm (Rouge, Verte, Jaune, Bleue)
-  4 × Boutons poussoirs (entrée joueur)
-  1 × Ruban LED Neopixel (8 LED - WS2812B)
-  1 × Écran LCD 16×2 (I2C)
-  Résistances 220 Ω (LED)



A19E3 Rappel du câblage

Module interface homme machine



A19E4 Un programme évolutif !

```
#Version 1: Affichage du jeu sur la console
#
#Version 2: Affichage du jeu sur un ecran LCD avec titre...
#
#Version 3: Utilisation du ruban Neopixel pour afficher le niveau de jeu par exemple
# Niveau 1 à 4: toutes les leds en vert
# Niveau 5 à 10: "" en jaune
# Niveau 10 et +: "" en rouge
#
#Version 4: Utilisation du ruban Neopixel comme bar graph...
# Niveau 1 à 4: leds 0-2 en vert
# Niveau 5 à 10: leds 0-2 en vert + leds 3-5 en jaune
# Niveau 10 et +: leds 0-2 en vert + leds 3-5 en jaune + leds 6-8 en rouge
```

A19E5 Version simplifiée (1/2)

```
from machine import Pin
import time
import random
```

```
#-----
# Declaration des boutons et des leds
#-----

# Definition des LEDs
leds = [
    Pin(4, Pin.OUT), #rouge
    Pin(2, Pin.OUT), #verte
    Pin(3, Pin.OUT), #bleu
    Pin(5, Pin.OUT)  #jaune
]

# Definition des boutons
boutons = [
    Pin(0, Pin.IN, Pin.PULL_UP), #bouton sous la led rouge
    Pin(1, Pin.IN, Pin.PULL_UP), #bouton sous la led verte
    Pin(11, Pin.IN, Pin.PULL_UP), #bouton sous la led bleu
    Pin(10, Pin.IN, Pin.PULL_UP) #bouton sous la led jaune
]

#-----
# Fonctions utiles pour simplifier la lisibilité du code
#-----

# Comme les leds sont branchées en "collecteur ouvert" leur commande est "inversée"
# On définit 2 fonctions "pour le cacher"
def led_on(index):
    leds[index].off()

def led_off(index):
    leds[index].on()

# Pour simplifier la lecture du code on regroupe toutes les commandes
# pour allumer la led pendant un instant dans une fonction
def allumer_led(index, duree=0.5):
    led_on(index)
    time.sleep(duree)
    led_off(index)
    time.sleep(0.2)
```

```
#-----
# Fonctions du jeu
#-----

# On stocke également la gestion du bouton dans une fonction
# pour simplifier la lecture du code
def attendre_bouton():
    while True:
        for i, bouton in enumerate(boutons):
            if bouton.value() == 0: # appui
                led_on(i)
                while bouton.value() == 0:
                    pass
                led_off(i)
                time.sleep(0.1)
                return i

def afficher_sequence():
    time.sleep(0.5)
    for note in sequence:
        allumer_led(note)

#-----
# Fonctions vides à compléter !!!!
#-----

def erreur():
    # pass indique qu'on ne fait rien dans cette fonction pour l'instant
    # et que c'est ok
    pass

def victoire():
    # pass indique qu'on ne fait rien dans cette fonction pour l'instant
    # et que c'est ok
    pass
```

A19E6 Version simplifiée (2/2)

```
#-----  
# Initialisation  
#-----  
  
# On éteint toutes les leds  
for i in range(4):  
    led_off(i)  
time.sleep(1)  
  
# On les rallume pour vérifier qu'elles sont toutes ok  
for i in range(4):  
    led_off(i)  
time.sleep(1)  
  
# On les re éteint pour commencer à jouer  
for i in range(4):  
    led_off(i)  
time.sleep(1)  
  
# On vide la liste qui contient la sequence de jeu  
# On va la construire pas à pas dans le code  
sequence = []
```

```
#-----  
# Programme principal  
#-----  
  
print("Jeu de Simon")  
  
while True:  
    # Ajoute une nouvelle étape  
    sequence.append(random.randint(0, 3))  
  
    # Message d'information sur la console de l'ordinateur  
    print("Niveau :", len(sequence))  
    afficher_sequence()  
  
    # Vérification de la saisie du joueur  
    perdu = False  
    for valeur_attendue in sequence:  
        entree = attendre_bouton()  
        if entree != valeur_attendue:  
            print("Perdu ! Score :", len(sequence) - 1)  
            erreur()  
            sequence = [] #on recommence, on efface la sequence  
            break  
        else:  
            print("Bravo !")  
            victoire()  
  
    # On attend un peu  
    time.sleep(1)
```


A19E7 Code LCD à réutiliser

```
from machine import Pin, I2C
from machine_i2c_lcd import I2cLcd
from time import sleep

#Declaration de l'ecran LCD
i2c = I2C(1, scl=Pin(15), sda=Pin(14), freq=400000)
addr = i2c.scan()[0]
print(hex(addr))
lcd = I2cLcd(i2c, addr, 2, 16)

while True:
    lcd.putstr(" Hello Raspi ! \n")
    lcd.putstr(" MJC 2025-2026 \n")
    sleep(1)
    #lcd.clear()
```

A19E8 Code Neopixel à réutiliser (1/2)

```
from neopixel import Neopixel
import time

# declaration du ruban de leds
NUM_LED = 8
PIN_NB = 13
leds = Neopixel(NUM_LED, 0, PIN_NB, "GRB")

# on definit la luminosite des leds / 255
leds.brightness(15)
# on eteint toute les leds
leds.clear()
# on affiche
leds.show()

while True:
    # on allume toutes les leds du sapin en rouge pendant 1 sec
    leds.fill((255,0,0))
    leds.show()
    time.sleep(1)
    # on allume toutes les leds du sapin en vert pendant 1 sec
    leds.fill((0,255,0))
    leds.show()
    time.sleep(1)
    # on allume toutes les leds du sapin en bleu pendant 1 sec
    leds.fill((0,0,255))
    leds.show()
    time.sleep(1)
```

A19E9 Code Neopixel à réutiliser (2/2)

```
from neopixel import Neopixel
import time

# declaration du ruban de leds
NUM_LED = 30
PIN_NB = 0
leds = Neopixel(NUM_LED, 0, PIN_NB, "GRB")

# declaration de quelques couleurs
couleur_rouge = (255,0,0)
couleur_vert = (0,255,0)
couleur_bleu = (0,0,255)
couleur_noir = (0,0,0)

# on definit la luminosite des leds
leds.brightness(15)
# on eteint toute les leds
leds.clear()
# on affiche
leds.show()

while True:
    # on allume tour a tour les leds du sapin en bleu
    for i in range (0,NUM_LED):
        leds.set_pixel(i,couleur_bleu)
        leds.show()
        time.sleep(0.1)
    leds.clear()
    leds.show()

    # on allume un bloc de leds
    leds.set_pixel_line(5,20,couleur_rouge)
    leds.show()
    time.sleep(2)
    leds.clear()
    leds.show()

    # on fait un gradient de couleur
    leds.set_pixel_line_gradient(5,20,couleur_vert,couleur_bleu)
    leds.show()
    time.sleep(2)
    leds.clear()
    leds.show()
```